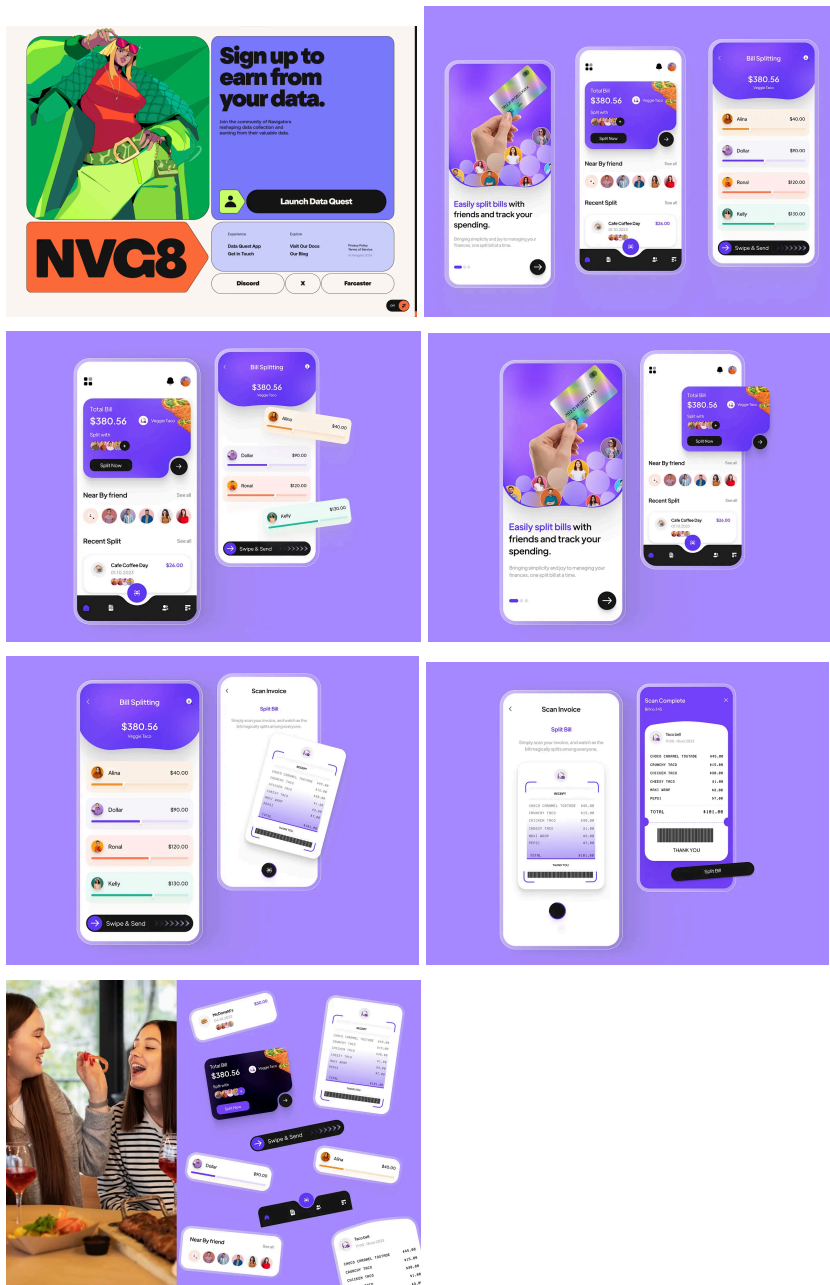




## UI/UX INSPO:

First image inspo - <https://www.footer.design/sites/navigate>

Remaining images inspo - <https://dribbble.com/shots/23313550-Bill-Splitting-App>

Combining the **bold, high-contrast, Gen-Z energy** of the first image (let's call it "Neo-Brutalism") with the **clean, rounded, glass-like card layouts** of the other images creates something entirely unique. It won't look like a boring banking app; it will look like a premium, trendy tool made exactly for college students.

Here is the complete, mapped-out blueprint for **Roomie** using this exact hybrid aesthetic.

## 1. The "Neon Bento" Design System

We are taking the layout from the bill-splitting UI but swapping out their soft gradients for the punchy, solid colors of the first image.

- **Background Base:** Warm Cream / Off-White (e.g., #FFFBF2).
  - **Primary Containers:** Matte Black (#1A1A1A) and Bright Lavender/Purple (#7B61FF).
  - **Accent Colors (For Buttons & Highlights):** Neon Lime Green (#74FF5A) and Punchy Orange (#FF5C39).
  - **Typography:** Massive, bold, extended sans-serif fonts for numbers/headers (like *Clash Display* or *Space Grotesk*), and a clean sans-serif for regular text.
  - **Shapes:** Extreme rounded corners (**rounded-[32px]**) for cards, combined with sharp, solid drop-shadows (no blurry shadows—solid black offset shadows to match Image 1's vibe).
- 

## 2. Complete App Wireframes & Features (V1)

### Screen 1: The "Roomie" Dashboard (Main Hub)

*Taking inspiration from the leftmost screen in your second upload, but tailored for PG life.*

- **Header:** "Roomie." in bold black, top left. Top right has a minimal settings icon.
- **Hero Card (Purple Container):** A large, bright purple rounded card.
  - Text: "Total PG Expenses (April)"
  - Amount: Huge white text **₹14,250**.
  - Visual: Overlapping circular avatars of all roommates (like in your screenshot).
- **Quick Action Row:** Two pill-shaped buttons.
  - Button 1 (Neon Green): "Mess Bill"
  - Button 2 (Orange): "Rent/Utility"
- **The Ledger List ("Who owes what"):**
  - Clean white or light gray cards for each roommate.
  - Incorporates the **progress bar/slider visual** from your third screenshot to show how much of their total monthly share they've paid.
  - *Action:* Tapping a roommate opens their detailed ledger.
- **Bottom Navigation:** A floating black pill at the bottom with three icons (Home, History, Settings) and a **prominent overlapping purple "Scan/Add" button** in the center (exactly like your screenshots).

### Screen 2: The "Split Now" Flow (Center Button Action)

*Triggered by the floating center button. A fluid bottom-sheet modal.*

- **Input Area:** A massive keypad (like a calculator) takes up the bottom half.
- **Top Area:** You type the amount (e.g., ₹450).

- **Tagging:** Below the amount, horizontal scroll of roommate avatars to select who was involved (default selects all).
- **The "Swipe & Send" Button:** Taking direct inspiration from your screenshots, instead of a boring "Submit" button, we use a custom **"Swipe to Split"** slider. You drag an arrow to the right, it haptics, and the bill is split.

### Screen 3: The "Settle Up" & UPI Deep Link Screen

*When you tap on a roommate who owes you money.*

- **Visual:** A clean, punchy card showing "Rahul owes ₹1,240".
- **Breakdown:** A mini-receipt view (inspired by your receipt scanning screenshot, but manually generated). It lists: *Mess (₹400), Wifi (₹200), Rent (₹640)*.
- **Action:** A massive Neon Green button at the bottom: **"Settle via UPI"**.
  - *Tech:* This triggers **expo-linking** to open GPay/PhonePe with the exact amount.

### Screen 4: Monthly Wrap-Up (Share Screen)

- **Visual:** Generates a beautiful "Digital Receipt" (exactly like the Taco Bell receipt in your last screenshot, but branded for your PG).
- **Action:** A black pill button saying "Share to WhatsApp".

---

## 3. The Animation Engine (60fps Polish)

To make it feel like a top-tier app, we will use **react-native-reanimated**.

- **The "Swipe & Send" Slider:** When adding an expense, dragging the slider will fill the track with Neon Green. On release, a satisfying spring animation shrinks the modal down and updates the dashboard.
- **Bento Card Mount:** When opening the app, the Hero Card and Ledger list don't just appear; they slide up from the bottom with a slight stagger (Hero Card first, then Roommate 1, Roommate 2, etc.) using a spring configuration.
- **Micro-interactions:** \* Tapping any button scales it down slightly (**scale: 0.95**).
  - Triggering the UPI button triggers a heavy haptic vibration (**expo-haptics**).

## 1. The "Zero-Friction" Onboarding & Multiplayer

- **Admin-First Setup:** The admin can download the app and start adding roommate names offline immediately, without needing an account. The app works flawlessly from second zero.
- **WhatsApp Deep-Link Invites:** The admin taps "Invite Roomies," generating a unique link sent via WhatsApp. When a roommate clicks it, they download the app, authenticate, and instantly join the specific "PG Ledger," syncing the offline data to the cloud.
- **Cloud Background Sync:** Built on a Local-First architecture (like Supabase + WatermelonDB/SQLite). The app writes data locally first for instant speed (no loading spinners), and silently syncs to the cloud whenever hostel Wi-Fi connects.

## 2. The Expense Logging Engine

- **PG-Specific Quick Templates:** One-tap categories tailored for Indian hostels: *Daily Mess, Monthly Rent, Electricity (Meter reading), Wifi, Maid/Water*. \* **The "Swipe-to-Split" Slider:** Instead of a boring "Save" button, users confirm expenses by dragging a sleek, neon-accented slider. It features satisfying haptic feedback (*expo-haptics*) upon completion.
- **Multi-Payer Simplification (The Math Engine):** Handles complex restaurant or grocery bills. E.g., The bill is ₹1500. Rahul pays ₹700, Aman pays ₹500, Admin pays ₹300. The app automatically calculates the "Net Balance" for everyone (who gets money back, who owes money) and updates the dashboard instantly.
- **The "Snap or Skip" Receipt Camera:** The moment an expense is swiped to split, a full-screen camera instantly appears. The admin can snap a photo of the bill/electricity meter for proof, or tap the prominent "Skip →" button if they are in a rush.

## 3. The Dashboard & UI/UX (Neon Bento Style)

- **Bento Box Layout:** Information is displayed in bold, high-contrast, heavily rounded cards (Matte Black, Bright Purple, Neon Green).
- **Monthly Burn Card:** A massive top card showing the total PG expenditure for the current month at a glance.
- **Live Roommate Progress Bars:** Instead of just showing numbers, each roommate's card has a visual slider bar showing how much of their total monthly share they have settled vs. what is pending.
- **60fps Spring Animations:** Buttery smooth transitions when opening modals or adding expenses, powered by *react-native-reanimated*.

## 4. Transparency & Dispute Resolution (The "Anti-Fight" Features)

- **The 'i' Explainer Bottom Sheet:** Next to any owed amount, an 'i' button opens a clean breakdown. It shows plain-English math: *"Total Bill ₹1500. Split 3 ways = ₹500 each. You paid ₹300. You are short -₹200. Rahul gets +₹200."*
- **Receipt Viewer:** If a photo was snapped during logging, it appears as a thumbnail inside the Explainer Sheet. Tapping it expands the original bill so roommates can verify the items.
- **"Flag This Expense" Button:** Roommates can tap a red warning button on a specific bill if the math looks wrong or they weren't present. It marks the expense as "Disputed" on the dashboard and notifies the admin to resolve it.

## 5. Settlement & Export

- **The Magic UPI Button:** The absolute core feature. Inside a roommate's "Owes" card, tapping "Settle via UPI" triggers a deep link opening GPay, PhonePe, or Paytm. It automatically pre-fills the exact amount and a note (e.g., "April Mess Dues").
- **Partial & Manual Cash Settlement:** Allows users to log partial payments or note that someone paid their dues in physical cash.
- **Digital "Taco Bell" Receipt Generator:** At the end of the month, the app generates a highly stylized, branded image summarizing the entire month's ledger.
- **One-Tap WhatsApp Export:** Send the generated Digital Receipt directly to the PG WhatsApp group as visual proof to close out the month.

## 6. Built-in Monetization

- **Aesthetic AdMob Integration:** A single, dark-themed banner ad that blends naturally into the bottom of the ledger list, generating passive income as the user base grows.
- **The ₹49 "Pro Unlock":** A manual upgrade path. The user pays ₹49 to your personal UPI, and you provide a 4-digit code. Typing it into the app removes ads and unlocks "Unlimited Groups" (for road trips or office lunches).

This list is your Bible for V1. It hits the perfect balance: it looks like a Gen-Z lifestyle app but processes complex multi-payer math like enterprise accounting software.

Here is exactly how we structure the app, from the cloud infrastructure down to the folder structure.

## 1. High-Level System Architecture (The "Local-First" Loop)

- **The Frontend:** React Native via Expo (using Expo Router for navigation).
- **The Local Database:** WatermelonDB (built on top of SQLite). It is explicitly designed for React Native offline-first apps and handles the complex math and UI updates at 60fps.
- **The Cloud Backend:** Supabase (PostgreSQL).
- **The Sync Engine:** WatermelonDB has a built-in sync function. When the app is online, it pushes local changes to Supabase and pulls any changes made by other roommates.
- **Asset Storage:** Supabase Storage (for hosting the receipt photos).

## 2. The Database Schema (The Math Engine)

To handle the complex multi-payer scenarios (e.g., Rahul pays ₹700, Aman pays ₹500, you pay ₹300), the database needs to be structured perfectly.

**Table 1: `users`**

- `id` (UUID, Primary Key)
- `name` (String)
- `phone_number` (String, Optional for admin-first setup)

**Table 2: `groups` (The PG Ledger)**

- `id` (UUID)
- `name` (String, e.g., "404 Boys PG")
- `admin_id` (UUID, linked to users)

**Table 3: `expenses` (The Bill)**

- `id` (UUID)
- `group_id` (UUID)
- `title` (String, e.g., "Groceries", "Mess")
- `total_amount` (Decimal)
- `receipt_image_url` (String, nullable)
- `status` (String: 'active', 'disputed', 'settled')
- `created_at` (Timestamp)

**Table 4: `expense_splits` (The Secret Sauce)** *This table maps exactly who paid what, and who owes what for every single bill.*

- `id` (UUID)

- `expense_id` (UUID)
- `user_id` (UUID)
- `paid_amount` (Decimal) -> *How much cash they actually put in.*
- `owed_amount` (Decimal) -> *Their actual share of the bill.*
- (Logic: If `paid_amount > owed_amount`, they get money back. If `owed_amount > paid_amount`, they owe money).

### 3. State & UI Architecture

- **Data State (The Numbers):** Handled entirely by WatermelonDB. When you query the database, it uses "Observables." This means if a background sync pulls down a new expense from Supabase, your React Native UI updates instantly without you writing any extra state logic.
- **App State (The UI):** Handled by **Zustand**. We use this for lightweight UI states that don't need a database.
  - E.g., `isCameraOpen`, `activeBottomSheet`, `selectedRoommateForSplit`.
- **Animations:** `react-native-reanimated` + `moti`.
- **Styling:** NativeWind (Tailwind for React Native) to paint the "Neon Bento" UI.

### 4. Folder Structure (Expo Router)

A clean structure ensures you don't get lost as the app grows.

Plaintext

roomie-app/

```

├── app/                                # Expo Router screens
│   ├── (tabs)/                         # Bottom navigation tabs
│   │   ├── _layout.tsx                 # Tab bar config
│   │   ├── index.tsx                  # The Main Dashboard
│   │   └── history.tsx                 # Past expenses list
│   ├── _layout.tsx                     # Global app layout & providers
│   ├── onboarding.tsx                 # Initial "Name your PG" screen
│   └── camera.tsx                     # Full-screen "Snap or Skip" camera
├── components/                         # Reusable UI parts
│   ├── BentoCard.tsx                  # The heavily rounded cards
│   ├── SwipeToSplit.tsx               # The custom swipe slider
│   └── ExplainerSheet.tsx             # The 'i' button bottom sheet
├── db/                                # WatermelonDB setup
│   ├── schema.ts                      # Local database schema
│   ├── sync.ts                        # Supabase sync logic
│   └── models/                        # Data models (User, Expense)
├── store/                             # Zustand state management
│   └── useAppStore.ts                 # UI state
├── utils/                             # Helper functions
│   ├── mathEngine.ts                 # Calculates who owes whom
│   └── upiLinking.ts                 # Generates the upi:// string

```

## 5. Native Integration Flow

1. **Adding an Expense:** User inputs ₹1500 -> Saves to WatermelonDB (`expenses` and `expense_splits` tables) -> UI updates instantly.
2. **Taking a Receipt Photo:** Triggers `expo-camera` -> Saves image locally to cache -> Uploads to Supabase Storage in the background -> Updates WatermelonDB with the public image URL.
3. **Settling Up:** User taps "Settle ₹200" -> Triggers `utils/upiLinking.ts`.
  - Code builds string:  
`upi://pay?pa=rahul@upi&pn=Rahul&am=200&cu=INR&tn=RoomieApp`
  - `Linking.openURL()` is called, prompting the OS to open PhonePe/GPay.
4. **Syncing:** The `db/sync.ts` file runs a background task every time the app opens or internet is restored, pushing local SQL rows to Supabase.

This architecture is industrial-grade but entirely manageable for a solo developer because Expo and WatermelonDB handle the heavy lifting.